DATABRICKS CERTIFIED DATA ENGINEER

Advanced Track — Study Guide

Chapter 7

Performance Optimization & Cluster Fine-Tuning

Spark internals · query optimization · Delta Lake layout · skew, shuffle, spill & serialization · choosing compute

Rebuilt and expanded for exam revision, with plain-language explanations, a glossary, a one-page cheat sheet, self-check questions, and official documentation links.

How This Chapter Fits Together

Chapter 7 moves from theory into performance engineering. It answers one big question in two halves: “why is my Spark job slow, and what do I actually do about it?” The first half looks inside a single query (architecture, the optimizer, data layout, and four classic code-level problems). The second half zooms out to the cluster itself (which compute type, which instance, Photon, Serverless). The map below mirrors the original lecture order.

- 1. Spark Architecture & the Spark UI — jobs, stages, tasks, driver, executors, cores
- 2. The Query Optimization Engine — Catalyst Optimizer & Adaptive Query Execution (AQE)
- 3. Designing the Foundation — what makes a query fast, the small-file problem, Auto Optimize
- 4. Data Skipping, Z-Ordering, Partitioning & Liquid Clustering
- 5. Table Statistics & Predictive Optimization
- 6. Code Optimization: Skew
- 7. Code Optimization: Shuffles
- 8. Code Optimization: Spill
- 9. Code Optimization: Serialization & UDFs
- 10. Fine-Tuning: Choosing the Right Cluster (types, spot, autoscaling, Photon, Serverless)
- 11. Fine-Tuning: Picking the Best Instance Type (the IFTTT framework)
- Glossary · One-Page Cheat Sheet · Self-Check Questions · Official Resource Links

1. Spark Architecture & Reading the Spark UI

Before you can optimize anything, you need a mental model of what actually happens when you run a Spark command. Every optimization technique later in this chapter is really just a way of controlling one of the pieces described below.

Jobs, Stages, and Tasks — the three levels of "work"

- **Job —** created every time you trigger an action (write, count(), collect(), display()). A job is the complete piece of work needed to satisfy that action.
- **Stage —** a job is broken into stages. A new stage boundary is created whenever data has to be shuffled (moved between partitions). Stages run strictly in order — Stage 2 cannot begin until Stage 1 has fully finished.
- **Task —** the smallest unit of work. One task processes exactly one partition of data. All the tasks inside a stage run in parallel, spread across the available cores.

Driver, Worker, Executor, Core — the four physical building blocks

- **Driver —** the "brain" of the application. It maintains the state of the Spark application, responds to your code, and analyzes/schedules work across the executors. There is exactly one driver per Spark application, no matter how many executors are attached.
- **Worker node —** a machine that hosts one or more executor processes.
- **Executor —** a process that holds a chunk of data (a Spark partition) and does the actual work. Each executor executes the tasks it's assigned and reports the results and its state back to the driver.
- **Core (a.k.a. slot / thread) —** the unit of parallelism inside an executor. Spark parallelizes at two levels — across executors, and again across the cores/slots within each executor. Each core runs one task at a time.

> **Common exam trap**
>
> A Spark partition is not the same thing as a disk partition. A Spark partition is simply the chunk of rows an executor is holding in memory for processing — it has nothing to do with how the underlying storage volume is partitioned.

The classroom analogy (from the lecture)

The course explains this with a simple picture: a class is asked to sort brown pieces out of candy bags. The whole classroom is the cluster. The teacher is the driver — handing out work and collecting results. Each table of students is an executor, and each individual student is a core. The pile of candy bags is your dataset, and each bag is a partition. When students at different tables have to exchange partial results with each other (for example, to add up a grand total across the whole class), that exchange of data between executors is exactly what Spark calls a shuffle — the topic of Section 7.

The lecture also uses this analogy to show why some work needs "stages": counting candies inside one bag can happen immediately and in parallel, but getting the grand total requires a second step (adding up everyone's sub-totals) that can only start once every table has finished its own count. That's the difference

between a job that completes in one stage and one that needs two — the second stage always waits for the first.

2. The Query Optimization Engine — Catalyst & Adaptive Query Execution

When you write a DataFrame or SQL query, Spark doesn't run it literally line-by-line the way a script does. It first builds a plan, optimizes that plan, and only then executes it. Understanding this pipeline explains why some rewrites of the "same" query run much faster than others.

How a query becomes an execution plan

1. Your DataFrame/SQL code is parsed into an unresolved Logical Plan.
2. Catalyst resolves table/column names against the Catalog, producing a (resolved) Logical Plan.
3. Catalyst applies rule-based and cost-based optimizations to produce an Optimized Logical Plan.
4. Spark generates one or more candidate Physical Plans and selects the cheapest one using a cost model.
5. Code generation turns the chosen physical plan into the actual distributed operations that run on the cluster.

> **What is the Catalyst Optimizer?**
>
> Catalyst is Spark SQL's built-in query optimizer — conceptually similar to the query planner inside a relational database. It rewrites your query into an equivalent but cheaper plan before touching any data (predicate pushdown, reordering filters, choosing join strategies, and so on). It only sees DataFrame/SQL/Dataset code — it cannot see into RDD code or the inside of a UDF, which is why both are treated as "optimization blockers" later in this chapter.

Adaptive Query Execution (AQE)

AQE (on by default since Spark 3.2) re-optimizes the plan while the query is actually running, using real statistics gathered at the end of each shuffle — instead of relying only on estimates made before any data was touched. It has four headline tricks:

- Dynamically switches join strategies — e.g. upgrading a Sort-Merge Join to a cheaper Broadcast Hash Join once AQE sees that one side of the join turned out to be small.
- Dynamically coalesces shuffle partitions — merges many tiny post-shuffle partitions into fewer, right-sized ones.
- Dynamically optimizes skew joins — automatically splits an oversized partition into several evenly sized pieces (see Section 6).
- Dynamically detects and propagates empty relations — skips work entirely on data that turns out to be empty.

Code optimization ground rules

- **Use DataFrames or SQL, not RDDs.** Catalyst can only optimize DataFrame/SQL/Dataset code. RDD code bypasses the optimizer completely, so you lose every automatic optimization described above.

- **Avoid unnecessary actions in production jobs.** count(), display(), and collect() each trigger a fresh pass over the data. Save them for when the result is actually needed — reading and writing files should usually be the only actions in a production pipeline.
- **Don't force everything onto the driver.** Single-threaded Python/pandas code runs entirely on the driver and defeats distributed processing. Use the Pandas API on Spark instead, so pandas-style syntax still executes across the whole cluster.

3. Designing the Foundation

This part of the chapter steps back and asks: independent of any specific tuning trick, what fundamentally makes one query faster than another on a big-data / MPP (Massively Parallel Processing) system like Databricks?

Four levers that decide query speed

- **Number of bytes read —** less physical data pulled from disk or network is always faster.
- **Query complexity / computation —** joins, aggregations, and window functions cost more CPU than a simple scan.
- **Number of files accessed —** every file open/close carries fixed overhead, so accessing many small files is slower than accessing a few large ones, even for the same total bytes.
- **Parallelism —** how well the work is spread across cores and executors — the defining property of an MPP system.

Common performance bottlenecks

- **The small file problem —** too many tiny files causes heavy per-file overhead and can even trigger cloud-provider I/O throttling.
- **Data skew —** one partition ends up much larger than the rest, so the whole stage waits on the single slowest task (full detail in Section 6).
- **Processing more data than necessary —** unlike traditional data lakes that often rewrite whole datasets, Databricks can scan only the required files and use data skipping to avoid touching irrelevant ones (Section 4).

Auto Optimize: fixing the small-file problem automatically

“Auto Optimize” is the umbrella name for two Delta Lake features that work together so you rarely need to manually compact files just to keep them a healthy size:

- **Optimized Write —** operates during the write itself: it dynamically resizes the Spark partitions being written based on the actual data, aiming for ~128 MB files right away.
- **Auto Compaction —** runs after the write job finishes, as a small follow-up job that checks whether files can be compacted further toward a standardized size (target ~128 MB, only triggered once a partition has enough small files to be worth it).

Note the terminology shift you'll see in current docs: Databricks now describes these as two separate settings (optimizeWrite and autoCompact) rather than one combined “auto optimize” feature — same mechanics, clearer naming.

4. Data Skipping, Z-Ordering, Partitioning & Liquid Clustering

This is the “data layout” half of performance tuning — arranging files on disk so that a query has to read as little as possible to get its answer.

Data Skipping

Delta Lake automatically records min/max statistics per file for (by default) the first 32 columns of a table. When a query filters on one of those columns, Spark checks the file-level statistics first, and skips any file whose min/max range cannot possibly contain the filter value — without ever opening the file. Some queries (e.g. SELECT MAX(col) FROM table) can be answered from the Delta transaction log's statistics alone, without reading any data files at all.

Z-Ordering

Z-Ordering physically co-locates similar values of a chosen column together in the same files. Because the min/max range of a Z-ordered column becomes much tighter per file, data skipping becomes dramatically more effective — a targeted query can open exactly the one file that could contain the value it's looking for, instead of having to check many files just in case.

> **Exam note**
>
> Databricks currently recommends Liquid Clustering over Z-Ordering (and over partitioning) for new tables — but Z-Ordering still exists, is still testable, and is still the right tool on older Databricks Runtime versions or tables that already use partitioning.

Delta Lake statistics — practical limits

- Controlled by the table property dataSkippingNumIndexedCols, default 32 columns.
- Filters are applied in this order: Partition filters → Data filters → Pushed filters.
- Timestamp and String column statistics aren't always precise — truncation can prevent an exact match, forcing a fallback to actually scanning the file.
- Avoid collecting statistics on long string columns — either move them past column 32, or lower dataSkippingNumIndexedCols so they're excluded.

What about partitioning?

Databricks' current guidance is refreshingly blunt: partitioning is “generally not recommended” for new tables, because it's so often misused or overused, leading to tiny files or skew. It still has a few legitimate use cases:

- Isolating data for separate schemas or tenants (single-table → multiplexed use cases).
- GDPR/CCPA-style workflows where you commonly need to delete an entire partition's worth of data.
- Use cases that need a hard physical boundary — e.g. an SCD Type 2 table partitioned on current vs. historical rows for better performance.

If you do partition, the rules of thumb are: choose a low-cardinality column, keep each partition between roughly 1 GB and 1 TB, partition tables that you expect to grow into the terabytes, and typically partition on a date column while Z-ordering on whatever columns show up most often in your WHERE clauses.

The downsides seen in practice: many small files, high metadata-operation overhead, slow reads, data skew, and inconsistent file sizes across partitions — exactly the problems Liquid Clustering was designed to remove.

Liquid Clustering

Liquid Clustering is Databricks' modern replacement for the partitioning + Z-Ordering combo. Instead of rigid partition boundaries, it clusters data by one or more chosen columns and lets Databricks intelligently decide which ranges of data to combine so file sizes stay roughly consistent — this adaptiveness is what “liquid” refers to.

Key benefits, straight from the lecture:

- Best performance out of the box, with clustering applied on write.
- The most consistent data skipping, and immunity to data skew.
- Minimal write amplification during table maintenance, thanks to true incremental optimization (no need to rewrite the whole table when clustering keys change).
- Row-level concurrency — simpler logic for concurrent writers, and reduced cognitive overhead for the engineer (no more agonizing over cardinality or column order when picking a clustering key).

Because Liquid Clustering targets a specific file size rather than a rigid boundary, a table clustered by, say, customer ID and date no longer has the “tiny partition for a small customer, huge partition for a big one” skew problem that hive-style partitioning produces.

5. Table Statistics & Predictive Optimization

Table Statistics

Beyond the file-level min/max stats used for data skipping, Databricks can also collect full column-level statistics with:

```
ANALYZE TABLE mytable COMPUTE STATISTICS FOR ALL COLUMNS
```

These statistics feed the cost-based optimizer and AQE, and specifically help Spark:

- Choose the right join type for a given pair of tables.
- Select the correct build side in a hash join (the smaller side should build the hash table).
- Calibrate join order in a multi-way join.

Predictive Optimization

Predictive Optimization uses Databricks' own telemetry about how a table is actually queried and written to decide when maintenance operations should run — automatically, on serverless compute, with no schedule for you to manage. For Unity Catalog managed tables it automatically runs:

- **OPTIMIZE —** to compact small files and, when Liquid Clustering is enabled, to reorganize data by the clustering keys.
- **VACUUM —** to remove data files that are no longer referenced, reducing storage costs.
- **ANALYZE —** to keep statistics fresh for query planning.

This is exactly the “set it and forget it” philosophy the lecture describes: once enabled at the account, catalog, or schema level, you stop scheduling and babysitting maintenance jobs, and Databricks decides the most cost-effective time to run them based on observed workload behavior.

6. Code Optimization: Skew

Data is normally read in as roughly balanced ~128 MB partitions. But after a transformation — especially a groupBy() or join() — one partition can end up holding disproportionately more records than the rest. A small amount of skew is harmless; a large amount means the whole stage has to wait for the single slowest task, and can even trigger spill or a hard-to-diagnose out-of-memory (OOM) error.

Worked example from the lecture: four partitions of city data are evenly sized on ingest. After aggregating by city, City D turns out to have twice the population of Cities A, B, and C. Its partition is now roughly twice as large, so it takes roughly twice as long to process and needs roughly twice the RAM — and the entire stage's runtime is dictated by that one oversized partition, not by the average.

Four fixes, roughly in order of preference

- **1. Adaptive Query Execution (AQE) —** on by default since Spark 3.0/3.2. AQE automatically splits (and replicates, if needed) an oversized partition into several evenly sized pieces during the shuffle. By default, a partition needs to be at least 256 MB and at least 5x the average partition size to be flagged as skewed — both thresholds are tunable.
- **2. Filter skewed values —** if the skew is caused by a specific value your use case doesn't need (a very common culprit: a huge number of NULLs in a join key), simply filtering it out solves the problem outright.
- **3. Skew hints —** a Databricks-specific SQL hint that tells Spark exactly which table/column (and optionally which values) are skewed, so it can build a better plan without waiting to detect it automatically. Easier to add than salting, but mostly relevant on older Spark 2.x-era environments where AQE isn't available.
- **4. Salting —** the fallback when nothing else works. Random integers are appended as suffixes to the skewed key, artificially spreading it across many smaller partitions during the shuffle. It works, but it's more invasive to implement than the other three options.

> **Exam note — the 2,000-partition ceiling**
>
> Once a job's shuffle partition count exceeds 2,000, Spark stops tracking the size of every individual shuffle block and keeps only the average size — so AQE can no longer detect skew. Fix it by either reducing the shuffle partition count below 2,000, or raising the relevant Spark configuration threshold above your actual partition count.

7. Code Optimization: Shuffles

A shuffle is the act of moving data from the output of one stage to the input of another — redistributing and reorganizing it across the network so that related rows end up on the same executor.

Narrow vs. wide transformations

- **Narrow transformation —** needs only one stage to complete (e.g. filter(), select(), withColumn()). No shuffle required — each partition can be processed independently.
- **Wide transformation —** needs two (or more) stages because data has to move between partitions. This is what triggers a shuffle. Common wide transformations: join(), distinct(), groupBy(), orderBy(), and even some actions like count().

Shuffle at a glance: map → shuffle → reduce

Conceptually this maps onto the classic MapReduce pattern: Stage 1 reads and maps/transforms the data. A shuffle then physically moves that mapped data across the network to whichever worker needs it for the next step. Stage 2 reduces (aggregates) the shuffled data into the final output, which then gets written out to a DataFrame or table.

Mitigation techniques

- Use fewer, larger workers to reduce network I/O — you still pay the cost of disk I/O, but data crosses fewer network hops.
- Use fast local disks (NVMe/SSD) to speed up shuffle reads and writes.
- Reduce the amount of data being shuffled: drop unnecessary columns, and filter out unneeded records as early (preemptively) as possible.
- Denormalizing datasets can help when the shuffle is rooted in a join — though AQE and Dynamic Partition Pruning (DPP) are making this less necessary than it used to be outside of Spark 3.
- Re-evaluate the join strategy: reordering the join, letting Spark dynamically switch strategies, or explicitly choosing between a Broadcast Hash Join (best when one side is small enough to send to every executor), a Shuffle Hash Join (the default under Photon), or a Sort-Merge Join (the default in open-source Spark).

A quick word on bucketing

Bucketing pre-sorts partitions at write time so a later Sort-Merge Join can skip its sort step. The lecture is blunt about this one, quoting Spark veteran Daniel Tomes: “If you are bucketing datasets, you are doing it wrong.” Bucketing is hard to get right, expensive to set up (especially on data that changes periodically), and Databricks generally doesn't consider it worthwhile for datasets under roughly 1–5 TB. The underlying cost is paid upfront on the assumption that it'll be repaid through many repeated joins between the same two tables.

8. Code Optimization: Spill

Spill is the act of moving data from RAM to disk, and later back into RAM again. It happens when a given partition is simply too large to fit in the memory allocated to it — Spark is forced into (potentially expensive) disk reads and writes just to free up local RAM, all to avoid the alternative: a dreaded out-of-memory (OOM) error that crashes the job.

Common causes

- Setting spark.sql.files.maxPartitionBytes too high (default is 128 MB) — oversized partitions no longer fit comfortably in RAM.
- Calling explode() on even a small array — it can multiply row counts far beyond what memory expects.
- A join() or crossJoin() that generates a lot of new rows.
- A join() or crossJoin() on a skewed key.
- A groupBy() on a low-cardinality column.
- countDistinct() and size(collect_set()) — both hold large intermediate structures in memory.
- Setting spark.sql.shuffle.partitions too low, or a careless use of repartition().

Reading spill in the Spark UI

Spill is represented by two paired values: Spill (Memory) — the size of the spilled partition as it existed in memory — and Spill (Disk) — the size of the same data once serialized to disk. The disk value is always smaller, thanks to compression gained naturally by serializing the data before writing it out.

The two numbers are only shown together, and only inside the details page for a single stage — in the Summary Metrics, the Aggregated Metrics by Executor table, the Tasks table, or the corresponding query details. This makes spill easy to miss, but there's a handy rule: when there is no spill at all, the spill columns don't even appear in the UI. If you see the columns, there is spill somewhere.

Mitigations

- Allocate a cluster with more RAM per core.
- Address the underlying data skew (Section 6) — a smaller, more balanced partition is less likely to spill.
- Manage the size of Spark partitions directly.
- Avoid expensive operations like explode() where you can.
- Reduce the amount of data preemptively — drop unneeded columns and rows before the expensive step, not after.

9. Code Optimization: Serialization

Spark SQL and DataFrame operations are already highly optimized by Catalyst. That optimization only applies to native operations, though — a user-defined function (UDF) is a black box to the optimizer. Every UDF must be serialized and distributed to each executor, and the parameters and return value of the UDF have to be converted for every single row of data passing through it.

Python UDFs take an even harder hit: on top of the generic UDF overhead above, the Python code itself has to be pickled, Spark has to instantiate a separate Python interpreter process inside every executor, and each row has to be converted back and forth between the JVM and Python — a third layer of cost that plain Scala/Java UDFs don't pay.

Mitigating serialization issues

- **Don't use UDFs at all, if you can avoid it.** The lecture's challenge: try to find a set of transformations that truly can't be expressed with Spark's built-in, continuously optimized, community-supported, higher-order functions.
- **If you must use Python, use Pandas UDFs (a.k.a. vectorized UDFs).** These use Apache Arrow to move data in batches rather than row by row, which is dramatically cheaper than stock Python UDFs. (Newer Arrow-native UDFs push this even further by skipping the pandas conversion step entirely.)
- **If you must use Scala, use Typed Transformations.** These are the Scala-side equivalent of preferring built-ins over a stock UDF.
- **Resist "porting" existing business logic into Spark via a UDF wrapper.** The lecture is direct on this: actually rewriting the logic natively in Spark almost always pays off, even though it takes more upfront effort than wrapping the old code in a UDF.

Why this matters to the optimizer

A UDF creates an analysis barrier for the Catalyst Optimizer. Catalyst cannot connect or reorder the code that runs before and after a UDF — the UDF itself is an opaque box, so any optimization opportunity that spans across the UDF boundary is lost, even if the code immediately before and after it is perfectly optimizable on its own.

10. Fine-Tuning: Choosing the Right Cluster

Everything so far has been about writing better code and laying out data well. This section zooms out to the compute layer itself — what kind of cluster should even be running that well-written code?

The three classic compute types

Compute type	Best for	Notes
All-Purpose Compute	Interactive development in notebooks	Shared by multiple users; recommended: autoscale + auto-stop; develop/test on a data subset
Jobs Compute	Scheduled/submitted production jobs	Created fresh per job, terminates on completion; single-user; good isolation; typically lower cost
SQL Warehouse	High-concurrency ad hoc SQL & BI serving	Photon built in; single-cluster or serverless multi-cluster scaling

Serverless Compute

Serverless is now Generally Available across all three compute types — All-Purpose, Jobs, and SQL Warehouses. Instead of you configuring instance types, the spot market, autoscaling thresholds, and idle shutdowns, Databricks manages the infrastructure layer entirely.

	Classic compute	Serverless compute
Startup time	~5–12 minutes	~15–30 seconds
Scaling	Manually configured autoscaling	Autoscales in seconds, automatically
Instance/spot management	You pick VM types & manage spot market	Fully automated
Pricing	Pay while the cluster is running	Pay only for what you actually use

The one thing that never changes

Serverless removes the infrastructure burden — but not the

	Classic compute	Serverless compute
need for good code and data layout. If your code has excessive shuffles, it will still be slow on serverless. Poor data layout (small files, no Liquid Clustering, no data skipping) won't magically fix itself. Photon still powers the query engine underneath. Databricks owns the infrastructure layer; you still own the code and data layer.		

Spot Instances

Spot instances let you use a cloud provider's spare, currently-unused VM capacity at a below-market rate, on the understanding that the provider can reclaim it if demand increases elsewhere. Great for ad-hoc or shared clusters; not recommended for jobs with mission-critical SLAs; never use spot for the driver.

Scenario	Recommended pattern
Non-mission-critical jobs	On-demand driver + spot workers (a reclaimed worker doesn't kill the job; a reclaimed driver would)

Scenario	Recommended pattern
Workflows with tight SLAs	Spot instances with automatic fallback to on-demand if spot capacity isn't available

Availability and savings vary a lot by instance family and region — the lecture's example: i3 instances aren't always great candidates, while r5d/r6d instances often show better savings and materially lower interruption rates. Always check the actual numbers for your region rather than assuming.

Autoscaling

Autoscaling dynamically resizes a cluster based on workload — it can run faster than a statically-sized, under-provisioned cluster, and it can reduce overall cost compared to a statically-sized cluster that's sized for the worst case. Setting the min/max worker range takes some experimentation.

- Ad-hoc usage or business analytics → allow a large variance in the range while you learn the workload.
- Production batch jobs → often don't need autoscaling at all, or only need a small buffer on the upper limit.
- Streaming → supported, and also available in Spark Declarative Pipelines.

Photon

Photon is Databricks' native, vectorized query engine, written in C++ — a drop-in accelerator for SQL and DataFrame workloads that requires no code changes. Headline figures the lecture calls out: ETL customers save up to ~40% on compute; roughly 6x better price/performance than other cloud data warehouses and a 2–3x reduction in query time vs. open-source Spark; and it works across exploration, ETL, big and small data, high concurrency, batch, and streaming — all with SQL, Python, Scala, R, or Java.

Cluster optimization recommendations (rules of thumb)

- Data science / data engineering development → all-purpose compute, autoscale + auto-stop enabled, develop and test on a subset of the data.
- Ingestion & ETL jobs → jobs compute, sized according to the job's SLA.
- Ad-hoc SQL analytics → (serverless) SQL warehouse, autoscale + auto-stop enabled.
- BI reporting → an isolated SQL warehouse, sized according to the BI workload's SLA.
- General best practices: enable spot instances on worker nodes; use the latest LTS Databricks Runtime when possible; use Photon for the best TCO where it applies; start with general-purpose VMs before testing memory- or compute-optimized types.

11. Fine-Tuning: Picking the Best Instance Types

Keep an open mind about machine families

- AWS: i3 isn't always the best choice — explore m7gd and r7gd for better processors and much more stable spot markets. Graviton (ARM) instances are worth trying first.
- Azure: try the Eav4, Dav4, and F-series before the L-series; the ACU (Azure Compute Unit) metric is a useful way to compare VM performance.
- GCP: the platform's own defaults are usually already reasonable.
- Network-optimized instances usually aren't necessary, but occasionally help specifically with Photon on bandwidth-heavy jobs.
- Enable caching if your workload benefits from it, regardless of which cloud you're on.

The IFTTT ("If This, Then That") decision framework

The chapter's practical answer to "how do I pick a machine?" is a short, repeatable checklist rather than guesswork:

Step	Question	If NO	If YES
1	Do you want to use Photon?	Move to Step 2	Start from the Photon-recommended VM family for your cloud
2	Is this an ETL job using joins / windows / group-bys / aggregations?	Use the lighter-workload instance suggestions for your cloud	Use the compute-oriented instance suggestions for heavy ETL
3	Run the job. Check the Spark UI for the longest-running query: is there spill?	Stop — this is good enough	Set spark.sql.shuffle.partitions to (largest shuffle-read stage size ÷ ~200MB), or set it to auto
4	Re-run with the updated shuffle partitions: still spilling?	Stop — this is good enough	Move to Step 5
5	Re-run with an updated instance type: still spilling?	Stop — this is good enough	Repeat the loop until spill disappears

Note: spill is much less impactful once Photon is in the picture, which is exactly why Step 1 (Photon or not) comes before Step 3 (checking for spill) in this framework.

> **Worth remembering for the exam**
>
> If you double the size of your cluster and the job finishes in half the time, the total cost is roughly the same — but you save real time. A "bigger" cluster isn't automatically a "more expensive" one if it also finishes proportionally faster.

Other rules of thumb from the chapter

- On the first run, set spark.sql.shuffle.partitions to roughly 2× the number of cores in the cluster.
- Keep the total memory available on any one machine under 128 GB.
- Aim for a core-to-data ratio of roughly 1 core per 128 MB–2 GB of data read (workload-dependent, with caveats).
- Avoid setting extra configs at first — don't carry over settings from a legacy platform unless there's a specific reason to.
- The file-system read data size shown in the Spark UI tells you how much data is actually being read from disk during a job — a quick sanity check on whether your cluster's size and setup make sense.

Don't forget the Event Log

Spot failures happen, and they slow things down. The event log should be the first thing you check when troubleshooting — it shows how the cluster resized during autoscaling (higher to lower worker counts and back), how many workers you actually needed, which VM types performed best, and whether you hit any errors or interruptions.

Glossary of Key Terms

Every term below appears in Chapter 7. Definitions are written for quick recall, not word-for-word textbook precision.

Term	What it means
Adaptive Query Execution (AQE)	Runtime re-optimization of a query plan using real statistics gathered after each shuffle; handles skew, coalesces small partitions, and switches join strategies dynamically. On by default since Spark 3.2.
All-Purpose Compute	Interactive cluster type used for notebook-based development and exploration; can be shared by multiple users.
Auto Compaction	A follow-up job that runs after a write completes, compacting small files toward a standardized size within a partition.
Autoscaling	Automatic growth/shrinkage of the number of worker nodes in a cluster based on current workload demand.
Broadcast Hash Join	A join strategy where the smaller table is sent in full to every executor, avoiding a shuffle entirely. Best when one side is small.
Bucketing	Pre-sorting data into a fixed number of buckets at write time to skip the sort step of a later Sort-Merge Join. Expensive to maintain and rarely recommended by Databricks.
Catalyst Optimizer	Spark SQL's built-in query optimizer; rewrites logical/physical plans for efficiency before execution. Cannot see through RDD code or UDFs.
Core (slot / thread)	The smallest unit of parallelism inside an executor; each core runs one task at a time.
Data Skew	A condition where one partition holds disproportionately more data than others, causing the whole stage to wait on the slowest task.
Data Skipping	Using file-level min/max statistics to avoid opening files that can't contain the data a query is filtering for.
Driver	The process that coordinates a Spark application: maintains state, analyzes code, and schedules tasks across executors. Exactly one per application.
Executor	A process on a worker node that holds a data partition and executes the tasks assigned to it.
Job	The complete unit of work triggered by a single Spark action (e.g. write, count(), collect()).
Jobs Compute	A cluster created specifically to run a scheduled/submitted job; terminates automatically when the job finishes.

Term	What it means
Liquid Clustering	Databricks' adaptive data-layout technique that replaces partitioning and Z-Ordering; clusters data incrementally on write without rigid boundaries.
MPP (Massively Parallel Processing)	An architecture (like Databricks/Spark) where computation is split and run in parallel across many machines.
Narrow Transformation	A transformation that needs only one stage and no shuffle (e.g. filter(), select()).
OOM Error (Out of Memory)	A job failure caused when there isn't enough RAM (and no room left to spill to disk) to hold the data being processed.
Optimized Write	A Delta Lake feature that dynamically resizes partitions during a write to target an efficient file size (default ~128 MB).
Photon	Databricks' native, vectorized query engine (written in C++) that accelerates SQL/DataFrame workloads with no code changes required.
Predictive Optimization	A Unity Catalog feature that automatically decides when to run OPTIMIZE, VACUUM, and ANALYZE on managed tables, based on observed usage.
Salting	Appending random integers to a skewed join key to artificially spread it across more partitions before a shuffle; the “last resort” skew fix.
Serialization	The process of converting a UDF (and the data passing through it) into a form that can be shipped to and executed on every executor.
Serverless Compute	A compute model where Databricks fully manages infrastructure provisioning, sizing, scaling, and shutdown; billed only for actual usage.
Shuffle	The act of moving data between partitions/executors across the network, required by wide transformations.
Skew Hint	A Databricks SQL hint that explicitly tells Spark which table/column/value combination is skewed, so it can build a better plan without needing to detect the skew itself.
Sort-Merge Join	A join strategy that sorts both sides on the join key before merging them; the default join strategy in open-source Spark.
Spill	Moving data from RAM to disk (and back) because a partition is too large to fit fully in memory.
Spot Instance	Spare cloud VM capacity offered at a discount; can be reclaimed by the provider at any time if demand increases.
SQL Warehouse	A compute type purpose-built for high-concurrency SQL analytics and BI serving, with Photon built in.

Term	What it means
Stage	A subdivision of a job; a new stage begins whenever a shuffle is required. Stages execute strictly in order.
Table Statistics	Column-level statistics (collected via ANALYZE TABLE) used by the cost-based optimizer and AQE to choose join types, build sides, and join order.
Task	The smallest unit of work in Spark; one task processes one partition of data.
TCO (Total Cost of Ownership)	The full cost of running a workload, including compute, storage, and operational/maintenance overhead — not just the sticker price of the VM.
UDF (User-Defined Function)	Custom code (Python/Scala/etc.) used inside a Spark query; opaque to the Catalyst Optimizer and generally the most expensive way to express a transformation.
Wide Transformation	A transformation that requires a shuffle and therefore spans two or more stages (e.g. join(), groupBy(), orderBy()).
Z-Ordering	A technique that physically co-locates similar values of a chosen column in the same files, making data skipping far more effective.

A term you asked about: Terraform

Terraform does not actually appear anywhere in your Chapter 7 slides — this chapter is entirely about Spark performance internals and cluster/instance selection, not infrastructure-as-code. Since you mentioned it, here's a quick, honest definition so you're not left wondering:

Terraform (bonus term, not part of this chapter)

Terraform is an open-source Infrastructure-as-Code (IaC) tool from HashiCorp. Instead of clicking through a cloud console, you describe the infrastructure you want (VMs, networks, and — via the official Databricks Terraform provider — things like Databricks workspaces, clusters, jobs, and Unity Catalog objects) in declarative configuration files, and Terraform figures out how to create, update, or tear down real cloud resources to match. It's typically part of the CI/CD or platform-engineering side of data engineering, alongside tools like Databricks Asset Bundles — not part of the query-performance and cluster-sizing material covered in this chapter.

One-Page Cheat Sheet: Symptom → Cause → Fix

Use this as your final-day-before-the-exam scan, or as a real diagnostic checklist on the job.

Symptom	Likely Cause	Fastest Fix
One task/partition runs far longer than the rest	Data skew	Let AQE handle it automatically; else filter out the skewed value, use a skew hint, or salt the key
Spill (Memory)/(Disk) columns appear in the Spark UI	A partition is too big for the allocated RAM	More RAM per core; fix data skew; manage partition size; avoid explode(); pre-filter/select fewer columns
Many tiny tasks, high per-file overhead	Small file problem	Optimized Write + Auto Compaction (Auto Optimize); scheduled OPTIMIZE; move to Liquid Clustering
Long shuffle read/write time	Expensive wide transformation	Fewer/larger workers; drop unneeded columns; filter early; pick the right join strategy
Query slow despite a small dataset	Row-at-a-time UDF (especially Python)	Replace with built-in functions, or use a Pandas/Arrow UDF instead of a stock Python UDF
AQE isn't fixing an obviously skewed job	Shuffle partition count exceeds 2,000	Reduce shuffle partitions below 2,000, or raise the AQE skew-detection threshold config
Idle cluster cost keeps creeping up	No auto-stop, or an oversized cluster	Enable autoscale + auto-stop; consider Serverless Compute for on-demand billing
Job fails or slows unpredictably on spot	Spot capacity reclaimed mid-job	On-demand driver + spot workers; enable fallback to on-demand for SLA-sensitive jobs
Table still slow to query after OPTIMIZE	Wrong or missing clustering strategy	Consider Liquid Clustering, or check that ANALYZE TABLE stats are current
Not sure which VM/instance to pick	No structured decision process	Run the IFTTT framework (Photon? → ETL-heavy? → check Spark UI for spill → iterate)

Self-Check Practice Questions

Answer these from memory first, then check the answer key at the end. All are drawn directly from this chapter's material.

1. In a Databricks cluster, how many drivers exist regardless of the number of executors?

- A) One per executor
- B) Exactly one
- C) One per worker node
- D) It's configurable

2. Which of the following is a wide transformation that requires a shuffle?

- A) filter()
- B) select()
- C) groupBy()
- D) withColumn()

3. By default, Adaptive Query Execution flags a partition as skewed once it is at least how much bigger than the average partition size (and at least 256 MB)?

- A) 2x
- B) 5x
- C) 10x
- D) 50x

4. What is the main functional difference between Liquid Clustering and the older partitioning + Z-Ordering approach?

- A) Liquid Clustering requires manual OPTIMIZE scheduling while Z-Ordering does not
- B) Liquid Clustering adapts the data layout incrementally on write and lets you change clustering keys without rewriting the table
- C) Z-Ordering is immune to data skew while Liquid Clustering is not
- D) Liquid Clustering only works on already-partitioned tables

5. By default, Delta Lake collects file-level min/max statistics on how many columns of a table?

- A) 8
- B) 16
- C) 32
- D) 64

6. Which compute type is purpose-built for high-concurrency ad hoc SQL analytics and BI serving?

- A) All-Purpose Compute
- B) Jobs Compute
- C) SQL Warehouse

- D) Single Node compute

7. What is “spill” in Spark?

- A) Data lost during a network failure
- B) Moving data from RAM to disk (and back) because a partition doesn't fit in memory
- C) Writing duplicate copies of data for fault tolerance
- D) The final step of writing an output table

8. Why are Python UDFs typically the slowest UDF option in Spark?

- A) They cannot be distributed across executors at all
- B) They require pickling, a separate Python interpreter per executor, and row-by-row conversion between Python and the DataFrame
- C) They are not supported on Databricks Runtime
- D) They always execute only on the driver

9. What does Predictive Optimization automatically manage for Unity Catalog managed tables?

- A) Cluster autoscaling policies
- B) OPTIMIZE, VACUUM, and ANALYZE operations
- C) Network security groups
- D) User authentication tokens

10. In the chapter's IFTTT instance-selection framework, what is the very first branching question?

- A) Is there data skew?
- B) Do you want to use Photon?
- C) Is the job an ETL job with joins/aggregations?
- D) What is the shuffle partition count?

11. Which of these is NOT one of the four common fixes for data skew discussed in this chapter?

- A) Adaptive Query Execution
- B) Salting the join keys
- C) Increasing spark.sql.files.maxPartitionBytes
- D) Filtering out the skewed values

12. Roughly how much faster does Serverless Compute start up compared to classic compute, per this chapter?

- A) No difference
- B) About 2x faster
- C) ~15–30 seconds vs. ~5–12 minutes
- D) Serverless never needs to start up

Answer Key

1B 2C 3B 4B 5C 6C 7B 8B 9B 10B 11C 12C

Note on Q11: increasing spark.sql.files.maxPartitionBytes is actually a common cause of spill (Section 8), not a fix for skew — a useful distractor to remember for the real exam.

Official Resource Links for Review

All links below point to Databricks' AWS documentation for readability; the same pages exist for Azure (learn.microsoft.com/en-us/azure/databricks/...) and GCP (docs.gcp.databricks.com/...) with the same path structure.

Spark architecture & the Spark UI

Debugging with the Spark UI: https://docs.databricks.com/aws/en/compute/troubleshooting/debugging-spark-ui

Diagnosing a long-running Spark job/stage: https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/long-spark-stage

Query optimization

Adaptive Query Execution (AQE): https://docs.databricks.com/aws/en/optimizations/aqe

Skew join optimization using skew hints: https://docs.databricks.com/aws/en/archive/legacy/skew-join

pandas / vectorized UDFs (Apache Arrow): https://docs.databricks.com/aws/en/udf/pandas

Data layout: small files, data skipping, clustering

Control data file size (Optimized Writes & Auto Compaction): https://docs.databricks.com/aws/en/delta/tune-file-size

Optimize data file layout (OPTIMIZE command): https://docs.databricks.com/aws/en/delta/optimize

Use Liquid Clustering for tables: https://docs.databricks.com/aws/en/tables/clustering

Predictive optimization for Unity Catalog managed tables: https://docs.databricks.com/aws/en/optimizations/predictive-optimization

Remove unused data files with VACUUM: https://docs.databricks.com/aws/en/sql/language-manual/delta-vacuum

Compute: clusters, Photon, Serverless

Compute overview (all-purpose, jobs, SQL warehouses, serverless): https://docs.databricks.com/aws/en/compute/

What is Photon?: https://docs.databricks.com/aws/en/compute/photon

Serverless compute for notebooks, jobs & pipelines: https://docs.databricks.com/aws/en/compute/serverless/

SQL warehouse types (serverless / pro / classic): https://docs.databricks.com/aws/en/compute/sql-warehouse/warehouse-types